Competitive Programming - Teambook

Datym1984 / competitive-programming

commit 29ec872 • 40 файлов

Содержание

1 Структуры данных **2**
- 1.1 cartesion tree with undefined key . . . 2
- 1.2 cartesion tree . . . 2
- 1.3 Li chao tree . . . 2
- 1.4 persistent segment tree . . . 3
- 1.5 sparce table . . . 3
- 1.6 template segment tree . . . 3

2 Динамическое программирование **4**
- 2.1 D&C min+ convolution . . . 4

3 Графы **4**
- 3.1 Dinic's algorithm . . . 4
- 3.2 ford-falkerson . . . 4
- 3.3 HLD . . . 4
- 3.4 Kuhn(maximal matching) . . . 5
- 3.5 LCA . . . 5
- 3.6 min cost max flow . . . 6
- 3.7 MIS, MVC . . . 6
- 3.8 online_bridges_search . . . 7
- 3.9 search BCC . . . 7
- 3.10 search centroid . . . 7

4 Теория чисел **7**
- 4.1 Diophantine equation . . . 7
- 4.2 Expanded Euclidean algo . . . 8
- 4.3 FFT double . . . 8
- 4.4 FFT modulo . . . 8
- 4.5 floor sum . . . 8
- 4.6 Long arithmetic (unfinished) . . . 8
- 4.7 Miller-Rabin . . . 8
- 4.8 ModInt . . . 9
- 4.9 Pollard . . . 9

5 Геометрия **9**
- 5.1 build tangents parallel line . . . 9
- 5.2 build tangents through point . . . 9
- 5.3 convex hull v2 . . . 10
- 5.4 convex hull . . . 10
- 5.5 is point in polygon . . . 10
- 5.6 Minkovski . . . 10

6 Строки **10**
- 6.1 Aho-Corasick . . . 10
- 6.2 hashes . . . 11
- 6.3 Manaker (may have mistakes) . . . 11
- 6.4 suffix array . . . 11
- 6.5 suffix automaton . . . 11
- 6.6 Z and prefix functions . . . 12

7 Разное **12**
- 7.1 otjig . . . 12
- 7.2 two sat . . . 12

1 Структуры данных

1.1 cartesion tree with undefined key

data_structures/cartesion tree with undefined key.cpp

```cpp
mt19937 gen(1984);

struct node {
    node *l=nullptr, *r=nullptr;
    int y;
    int sz=1, val, sm;
    node(int val) : val(val), sm(val), y(gen()) {}
};

int get_sz(node *v) {
    return v == nullptr ? 0 : v->sz;
}

int get_sm(node *v) {
    return v == nullptr ? 0 : v->sm;
}

void upd(node *v) {
    if (v == nullptr) return;
    v->sz = get_sz(v->l) + 1 + get_sz(v->r);
    v->sm = get_sm(v->l) + v->val + get_sm(v->r);
}

pair<node*, node*> splitk(node *v, int k) {
    if (v == nullptr) return mp(nullptr, nullptr);
    int lsz = get_sz(v->l);
    if (lsz >= k) {
        auto [v1, v2] = splitk(v->l, k);
        v->l = v2;
        upd(v);
        return mp(v1, v);
    }
    auto [v1, v2] = splitk(v->r, k - lsz - 1);
    v->r = v1;
    upd(v);
    return mp(v, v2);
}

node* merge(node *v1, node *v2) {
    if (v1 == nullptr) return v2;
    if (v2 == nullptr) return v1;
    if (v1->y < v2->y) {
        v2->l = merge(v1, v2->l);
        upd(v2);
        return v2;
    }
    v1->r = merge(v1->r, v2);
    upd(v1);
    return v1;
}

node* insert(node *v, int pos, int val) {
    auto [v1, v3] = splitk(v, pos);
    node* v2 = new node(val);
    return merge(merge(v1, v2), v3);
}

node* erase(node *v, int pos) {
    auto [v1, v23] = splitk(v, pos);
    auto [v2, v3] = splitk(v23, 1);
    delete v2;
    return merge(v1, v3);
}

pair<node*, int> get(node *v, int l, int r) {
    if (v == nullptr) return mp(v, 0);
    auto [v1, v23] = splitk(v, l);
    auto [v2, v3] = splitk(v23, r - l + 1);
    int res = v2->sm;
    return mp(merge(merge(v1, v2), v3), res);
}

void del(node *v) {
    if (v == nullptr) return;
    del(v->l), del(v->r);
    delete v;
}
```

1.2 cartesion tree

data_structures/cartesion tree.cpp

```cpp
mt19937 gen(228);

struct node {
    node *l=nullptr, *r=nullptr;
    int x, y;
    int mn=0, mx=0;

    node(int val) : x(val), mn(val), mx(val), y(gen()) {}
};

int get_max(node *v) {
    return v == nullptr ? -inf : v->mx;
}

int get_min(node *v) {
    return v == nullptr ? inf : v->mn;
}

void update(node *v) {
    if (v == nullptr) return;
    v->mx = max({get_max(v->l), v->x, get_max(v->r)});
    v->mn = min({get_min(v->l), v->x, get_min(v->r)});
}

pair<node*, node*> split(node *v, int val) {
    if (v == nullptr) return mp(nullptr, nullptr);
    if (v->x < val) {
        auto [v1, v2] = split(v->r, val);
        v->r = v1;
        update(v), update(v2);
        return mp(v, v2);
    }
    auto [v1, v2] = split(v->l, val);
    v->l = v2;
    update(v1), update(v);
    return mp(v1, v);
}

node* merge(node *v1, node *v2) {
    if (v1 == nullptr) return v2;
    if (v2 == nullptr) return v1;
    if (v1->y < v2->y) {
        v2->l = merge(v1, v2->l);
        update(v2);
        return v2;
    }
    v1->r = merge(v1->r, v2);
    update(v1);
    return v1;
}

bool find(node *v, int val) {
    if (v == nullptr) return false;
    if (v->x == val) return true;
    if (v->x < val)
        return find(v->r, val);
    return find(v->l, val);
}

node* insert(node *v, int val) {
    if (find(v, val)) return v;
    auto [v1, v3] = split(v, val);
    node *v2 = new node(val);
    return merge(merge(v1, v2), v3);
}

node* erase(node *v, int val) {
    if (!find(v, val)) return v;
    auto [v1, v23] = split(v, val);
    auto [v2, v3] = split(v23, val + 1);
    return merge(v1, v3);
}

pair<node*, int> nxt(node *v, int val) {
    auto [v1, v2] = split(v, val + 1);
    int res = get_min(v2);
    return mp(merge(v1, v2), res);
}

pair<node*, int> prv(node *v, int val) {
    auto [v1, v2] = split(v, val);
    int res = get_max(v1);
    return mp(merge(v1, v2), res);
}
```

1.3 Li chao tree

data_structures/Li chao tree.cpp

```cpp
const ll inf = 2e18 + 10;
const int c = 1e9 + 10;

struct line {
    ll k=0, b=-inf;
    int idx=-1;

    line() = default;
    line(ll k, ll b, int idx) : k(k), b(b), idx(idx) {}

    ll operator[](ll x) {
        return k * x + b;
    }
};

struct li_chao {
    struct node {
        node *lc=nullptr, *rc=nullptr;
        line ln;
    };

    node *root = new node();

    ~li_chao() {
        del(root);
    }

```

```
    void del(node *v) {
        if (v == nullptr) return;
        del(v->lc), del(v->rc);
        delete v;
    }

    void expand(node *v) {
        if (v->lc == nullptr)
            v->lc = new node();
        if (v->rc == nullptr)
            v->rc = new node();
    }

    void add_line(node *v, int l, int r, line ln) {
        int m = (l + r) / 2;
        if (v->ln[m] < ln[m])
            swap(v->ln, ln);
        if (l + 1 == r) return;
        expand(v);
        if (ln[l] > v->ln[l]) {
            add_line(v->lc, l, m, ln);
        }
        if (ln[r] > v->ln[r]) {
            add_line(v->rc, m, r, ln);
        }
    }

    line get(node *v, int l, int r, ll x) {
        if (v == nullptr) return line();
        if (l + 1 == r)
            return v->ln;
        int m = (l + r) / 2;
        line res;
        if (x < m)
            res = get(v->lc, l, m, x);
        else
            res = get(v->rc, m, r, x);
        return (res[x] > v->ln[x] ? res : v->ln);
    }
};
```

1.4 persistent segment tree

data_structures/persistent segment tree.cpp

```
//add to range and get sum on [:), can roll back to previous versions
struct node {
    ll sm=0, p=0;
    node *lc=nullptr, *rc=nullptr;
    node() = default;
    node(ll sm, ll p, node *lc, node *rc) :
        sm(sm), p(p), lc(lc), rc(rc) {}
    node(node *l, node *r) {
        sm = l->sm + r->sm;
        lc = l, rc = r;
    }

    node* apply(int l, int r, ll val) {
        node *v2 = new node(*this);
        v2->sm += val * (r - l);
        v2->p += val;
        return v2;
    }

    void push(int l, int r) {
        int m = (l + r) / 2;
        node *vl = lc->apply(l, m, p);
        node *vr = rc->apply(m, r, p);
        lc = vl, rc = vr;
        p = 0;
    }

    void expand() {
        if (lc == nullptr)
            lc = new node();
        if (rc == nullptr)
            rc = new node();
    }

    node* upd(int l, int r, int L, int R, ll val) {
        if (r <= L || R <= l || L >= R)
            return this;
        if (L <= l && r <= R)
            return apply(l, r, val);
        expand();
        push(l, r);
        int m = (l + r) / 2;
        node *vl = lc->upd(l, m, L, R, val);
        node *vr = rc->upd(m, r, L, R, val);
        return new node(vl, vr);
    }

    ll get(int l, int r, int L, int R) {
        if (r <= L || R <= l || L >= R)
            return 0ll;
        if (L <= l && r <= R)
            return sm;
        expand();
        push(l, r);
        int m = (l + r) / 2;
        return lc->get(l, m, L, R) + rc->get(m, r, L, R);
    }
};

template<typename T>
node* build(int l, int r, vector<T> &A) {
    if (l + 1 == r)
        return new node(-A[l], 0, nullptr, nullptr);
    int m = (l + r) / 2;
    return new node(build(l, m, A), build(m, r, A));
}
```

1.5 sparce table

data_structures/sparce table.cpp

```
struct ST {
    const int k = 20;
    vector<int> lg;
    vector<vector<int>> t;

    void build(vector<int> &A) {
        int n = (int) A.size();
        lg.resize(n + 1);
        t.resize(k, vector<int> (n));
        for (int i = 0; i < n; i++)
            t[0][i] = A[i];
        for (int i = 1; i < k; i++) {
            for (int j = 0; j + (1 << i) <= n; j++) {
                t[i][j] = min(t[i - 1][j], t[i - 1][j + (1 << (i - 1))]);
            }
        }

        for (int i = 2; i <= n; i++)
            lg[i] = lg[i / 2] + 1;
    }

    ST(vector<int> A) {
        build(A);
    }

    int get(int l, int r) {//return min[l:r)
        assert(l < r);
        int sz = r - l;
        return min(t[lg[sz]][l], t[lg[sz]][r - (1 << lg[sz])]);
    }
};
```

1.6 template segment tree

data_structures/template segment tree.cpp

```
struct segtree {
    struct node {

    };
    vector<node> t;

    segtree(int n) {
        t.resize(n * 4);
    }

    void apply(int v, int l, int r, int val) {

    }

    void push(int v, int l, int r) {

    }

    node merge(node v1, node v2) {

    }

    template<class elem>
    void build(int v, int l, int r, vector<elem> &A) {
        if (l + 1 == r) {
            apply(v, l, r, A[l]);
            return;
        }
        int m = (l + r) / 2;
        build(v * 2 + 1, l, m, A);
        build(v * 2 + 2, m, r, A);
        t[v] = merge(t[v * 2 + 1], t[v * 2 + 2]);
    }

    node get(int v, int l, int r, int L, int R) {//get on subsegment[l:r)
        if (r <= L || R <= l)
            return node();
        if (L <= l && r <= R)
            return t[v];
        push(v, l, r);
        int m = (l + r) / 2;
        return merge(get(v * 2 + 1, l, m, L, R), get(v * 2 + 2, m, r, L, R));
    }

    void upd(int v, int l, int r, int L, int R, int val) {//update on subsegment[l:r)
        if (r <= L || R <= l)
            return;
```

```
        if (L <= l && r <= R) {
            apply(v, l, r, val);
            return;
        }
        push(v, l, r);
        int m = (l + r) / 2;
        upd(v * 2 + 1, l, m, L, R, val);
        upd(v * 2 + 2, m, r, L, R, val);
        merge(t[v * 2 + 1], t[v * 2 + 2]);
    }
};
```

2 Динамическое программирование

2.1 D&C min+ convolution

dp/D&C min+ convolution.cpp

```
//A - concave vector i.e. A[i] - A[i - 1] <= A[i + 1] - A[i]
//B - arbitrary sequence
//C[i] = min<j>(B[j]+A[i-j])
//idea: C[i] - increasing
//solution in O(n*log(n))
vector<int> min_plus_convolution(int n, int m, vector<int> A, vector<int> B) {
  vector<int> C(n + m - 1);

  auto devide = [&](auto &&self, int l, int r, int opt_l, int opt_r)->void {
      if (l > r) return;
      int mid = (l + r) / 2;
      int opt_mid = max(mid - n + 1, opt_l);
      for (int i = max(mid - n + 1, opt_l); i <= min(mid, opt_r); i++) {
          if (B[i] + A[mid - i] < B[opt_mid] + A[mid - opt_mid])
              opt_mid = i;
      }

      C[mid] = opt_mid;
      self(self, l, mid - 1, opt_l, opt_mid);
      self(self, mid + 1, r, opt_mid, opt_r);
  };

  devide(devide, 0, n + m - 2, 0, m - 1);
  for (int i = 0; i < n + m - 1; i++)
      C[i] = B[C[i]] + A[i - C[i]];
  return C;
}
```

3 Графы

3.1 Dinic's algorithm

graphs/Dinic's algorithm.cpp

```
struct Dinic {
    struct edge {
        int v, u, c, f;
        edge() = default;
        edge(int v2, int u2, int c2, int f2) : v(v2), u(u2), c(c2), f(f2) {}
    };

    static const int INF = 1e9;
    vector<int> level, ptr;
    vector<vector<int>> g;
    vector<edge> edges;

    void add_edge(int v, int u, int c) { //add directed edge
        g[v].push_back((int) edges.size());
        edges.emplace_back(v, u, c, 0);
        g[u].push_back((int) edges.size());
        edges.emplace_back(u, v, 0, 0);
    }

    Dinic(int n) {//initialization
        level.resize(n), ptr.resize(n);
        g.resize(n);
    }

    bool bfs(int s, int t) {
        fill(all(level), -1);
        level[s] = 0;
        queue<int> Q;
        Q.push(s);
        while (!Q.empty()) {
            int v = Q.front();
            Q.pop();
            for (int id: g[v]) {
                int u = edges[id].v ^ edges[id].u ^ v;
                if (level[u] == -1 && edges[id].c - edges[id].f > 0) {
                    level[u] = level[v] + 1;
                    Q.push(u);
                }
            }
        }
        return level[t] != -1;
    }

    int dfs(int v, int t, int f) {
        if (f == 0) return 0;
        if (v == t) return f;
        int pushed = 0;
        for (; ptr[v] < g[v].size(); ptr[v]++) {
            int id = g[v][ptr[v]];
            int u = edges[id].v ^ edges[id].u ^ v;
            if (level[u] != level[v] + 1)
                continue;
            int f2 = dfs(u, t, min(f, edges[id].c - edges[id].f));
            if (f2 != 0) {
                edges[id].f += f2;
                edges[id ^ 1].f -= f2;
                pushed += f2;
                if (pushed == f)
                    return f;
            }
        }
        return pushed;
    }

    int MaxFlow(int s, int t) {//get maximal flow from s to t
        int flow = 0;
        while (bfs(s, t)) {
            fill(all(ptr), 0);
            while (int pushed = dfs(s, t, INF))
                flow += pushed;
        }
        return flow;
    }
};
```

3.2 ford-falkerson

graphs/ford-falkerson.cpp

```
struct FordFalkerson {
    const int INF = 1e9;
    vector<vector<int>> g;
    vector<vector<int>> f, c;

    FordFalkerson(int n) {
        g.resize(n);
        f.resize(n, vector<int> (n));
        c.resize(n, vector<int> (n));
    }

    void add_edge(int v, int u, int _c) {
        g[v].push_back(u);
        c[v][u] += _c;
        g[u].push_back(v);
    }

    int dfs(int v, int t, int cur_flow, vector<bool> &used) {
        if (v == t)
            return cur_flow;
        used[v] = true;
        for (int u : g[v]) {
            if (!used[u] && c[v][u] - f[v][u] > 0) {
                int flow = dfs(u, t, min(cur_flow, c[v][u] - f[v][u]), used);
                if (flow > 0) {
                    f[v][u] += flow;
                    f[u][v] -= flow;
                    return flow;
                }
            }
        }
        return 0;
    }

    int MaxFlow(int s, int t) {
        int flow = 0, new_flow;
        vector<bool> used(g.size());
        while (new_flow = dfs(s, t, INF, used)) {
            flow += new_flow;
            fill(all(used), false);
        }
        return flow;
    }
};
```

3.3 HLD

graphs/HLD.cpp

```
// 1) set val in v
// 2) search max in path from v to u v->...->u
struct Segtree {
    vector<int> t;

    Segtree(int n = 0) : t(n * 4) {}

    void resize(int n) {
        t.resize(n * 4);
    }

    void upd(int v, int l, int r, int idx, int val) {
        if (l + 1 == r) {
            t[v] = val;
            return;
        }
        int m = (l + r) / 2;
        if (idx < m)
```

```
            upd(v * 2 + 1, l, m, idx, val);
        else
            upd(v * 2 + 2, m, r, idx, val);
        t[v] = max(t[v * 2 + 1], t[v * 2 + 2]);
    }

    int get(int v, int l, int r, int L, int R) {
        if (r <= L || R <= l)
            return 0;
        if (L <= l && r <= R)
            return t[v];
        int m = (l + r) / 2;
        return max(get(v * 2 + 1, l, m, L, R), get(v * 2 + 2, m, r
    , L, R));
    }
};

int n;
vector<int> A, tin, tout;
vector<int> head, sz, par;
vector<vector<int>> g;
Segtree segtree;
int timer = 0;

void dfs(int v, int p) {
    par[v] = p;
    sz[v] = 1;
    for (int i = 0; i + (v != p) < g[v].size(); i++) {
        if (g[v][i] == p)
            swap(g[v][i], g[v].back());
        int u = g[v][i];
        dfs(u, v);
        sz[v] += sz[u];
        if (sz[u] > sz[g[v][0]])
            swap(g[v][0], g[v][i]);
    }
    if (v != p)
        g[v].pop_back();
}

void build_hld(int v) {
    tin[v] = timer++;
    segtree.upd(0, 0, n, tin[v], A[v]);
    for (int i = 0; i < g[v].size(); i++) {
        int u = g[v][i];
        if (i == 0)
            head[u] = head[v];
        else
            head[u] = u;
        build_hld(u);
    }
    tout[v] = timer;
}

bool is_ancestor(int p, int v) {
    return tin[p] <= tin[v] && tout[p] >= tout[v];
}

int get_hld(int v, int u) {
    if (tin[v] > tin[u])
        swap(v, u);
    int ans = 0;
    int cnt = 0;
    while (!is_ancestor(head[u], v)) {
        int res = segtree.get(0, 0, n, tin[head[u]], tin[u] + 1);
        ans = max(ans, res);
        u = par[head[u]];

        ++cnt;
        assert(cnt <= 100);
    }
    cnt = 0;
    while (!is_ancestor(head[v], u)) {
        int res = segtree.get(0, 0, n, tin[head[v]], tin[v] + 1);
        ans = max(ans, res);
        v = par[head[v]];

        ++cnt;
        assert(cnt <= 100);
    }
    int res = segtree.get(0, 0, n, min(tin[u], tin[v]), max(tin[u
    ], tin[v]) + 1);
    ans = max(ans, res);
    return ans;
}
```

3.4 Kuhn(maximal matching)

graphs/Kuhn(maximal matching).cpp

```
struct matching {
    vector<vector<int>> g;
    vector<int> c, used2;
    vector<bool> used;
    int n;

    matching(int n) : n(n) {
        used.resize(n);
        used2.resize(n);
        c.resize(n, -1);
        g.resize(n);
    }

    void add_edge(int v, int u) {
        g[v].push_back(u);
        g[u].push_back(v);
    }

    bool kuhn(int v, int num) {
        if (used2[v] == num) return false;
        used2[v] = num;
        for (int u : g[v]) {
            if (c[u] == -1) {
                c[u] = v;
                return true;
            }
        }
        for (int u : g[v]) {
            if (kuhn(c[u], num)) {
                c[u] = v;
                return true;
            }
        }
        return false;
    }

    void dfs(int v, int col, int &num) {
        used[v] = true;
        if (col == 0) {
            kuhn(v, num++);
        }
        for (int u : g[v]) {
            if (!used[u])
                dfs(u, 1 - col, num);
        }
    }

    vector<pair<int, int>> solve() {
        int num = 1;
        for (int v = 0; v < n; v++) {
            if (!used[v])
                dfs(v, 0, num);
        }
        vector<pair<int, int>> ans;
        for (int i = 0; i < n; i++) {
            if (c[i] != -1)
                ans.emplace_back(i, c[i]);
        }
        return ans;
    }
};
```

3.5 LCA

graphs/LCA.cpp

```
struct LCA {
    const int k = 20;// (1 << (k + 1)) >= n !!!
    int t = 0;
    vector<vector<int>> g;
    vector<vector<int>> up;
    vector<int> tin, tout, depth;

    void dfs(int v, int p) {
        tin[v] = t++;
        up[v][0] = p;
        for (int i = 1; i < k; i++)
            up[v][i] = up[up[v][i - 1]][i - 1];
        for (int u : g[v]) {
            if (u == p) continue;
            depth[u] = depth[v] + 1;
            dfs(u, v);
        }
        tout[v] = t;
    }

    LCA(vector<vector<int>> g, int root=0) : g(g) {
        int n = (int) g.size();
        up.resize(n, vector<int> (k));
        tin.resize(n), tout.resize(n), depth.resize(n);
        dfs(root, root);
    }

    int get_lca(int v, int u) {
        auto is_par = [this](int v, int u){
            return tin[v] <= tin[u] && tout[v] >= tout[u];
        };
        if (is_par(v, u)) return v;
        if (is_par(u, v)) return u;
        for (int i = k - 1; i >= 0; i--) {
            int v2 = up[v][i];
            if (!is_par(v2, u))
                v = v2;
        }
        return up[v][0];
    }

    int get_dist(int v, int u) {
        int lca = get_lca(v, u);
        return depth[v] + depth[u] - depth[lca] * 2;
    }
};
```

3.6 min cost max flow

graphs/min cost max flow.cpp

```cpp
// search min cost max flow in O(V*E + F*E*log(V)), when F - max flow
struct MCMF {
    struct edge {
        int v, u;
        int f, c, w;

        edge(int v, int u, int f, int c, int w) : v(v), u(u), f(f), c(c), w(w) {}
    };

    int n;
    vector<vector<int>> g;
    vector<edge> e;
    vector<ll> P;

    MCMF(int n) : n(n), g(n), P(n) {}

    void add_edge(int v, int u, int c, int w) {
        g[v].push_back(e.size());
        e.emplace_back(v, u, 0, c, w);
        g[u].push_back(e.size());
        e.emplace_back(u, v, 0, 0, -w);
    }

    void ford_belman(int s) {
        fill(all(P), 1e18);
        P[s] = 0;
        bool changed = false;
        for (int i = 0; i < n; i++) {
            changed = false;
            for (int j = 0; j < e.size(); j++) {
                if (e[j].c - e[j].f == 0) continue;
                int v = e[j].v, u = e[j].u;
                if (P[v] + e[j].w < P[u]) {
                    P[u] = P[v] + e[j].w;
                    changed = true;
                }
            }
            if (!changed)
                break;
        }
    }

    pair<ll, ll> dijkstra(int s, int t) {
        vector<ll> dist(n, 1e18);
        dist[s] = 0;
        set<pair<ll, int>> Q;
        Q.emplace(0, s);
        vector<int> par(n, -1);
        while (!Q.empty()) {
            int v = Q.begin()->second;
            Q.erase(Q.begin());
            for (int i : g[v]) {
                if (e[i].c - e[i].f == 0) continue;
                int u = e[i].u;
                ll w = P[v] + e[i].w - P[u];
                if (dist[v] + w < dist[u]) {
                    Q.erase({dist[u], u});
                    dist[u] = dist[v] + w;
                    Q.emplace(dist[u], u);

                    par[u] = i;
                }
            }
        }
        ll cost = (dist[t] - P[s] + P[t]);
        for (int v = 0; v < n; v++)
            P[v] += dist[v];

        if (par[t] == -1)
            return mp(0, 0);
        ll flow = 1e18;
        int u = t;
        while (u != s) {
            flow = min<ll>(flow, e[par[u]].c - e[par[u]].f);
            u = e[par[u]].v;
        }
        cost *= flow;
        u = t;
        while (u != s) {
            e[par[u]].f += flow;
            e[par[u] ^ 1].f -= flow;
            u = e[par[u]].v;
        }
        return mp(flow, cost);
    }

    pair<ll, ll> min_cost_max_flow(int s, int t) {
        ll flow = 0, cost = 0;
        ford_belman(s);
        while (true) {
            auto [f, c] = dijkstra(s, t);
            flow += f;
            cost += c;
            if (f == 0)
                break;
        }
        return mp(flow, cost);
    }
};
```

3.7 MIS, MVC

graphs/MIS, MVC.cpp

```cpp
struct matching {
    vector<vector<int>> g;
    vector<int> c, used2;
    vector<bool> used;
    vector<int> part;
    int n;

    matching(int n) : n(n) {
        used.resize(n);
        used2.resize(n);
        c.resize(n, -1);
        g.resize(n);
        part.resize(n, -1);
    }

    void add_edge(int v, int u) {
        g[v].push_back(u);
        g[u].push_back(v);
    }

    bool kuhn(int v, int num) {
        if (used2[v] == num) return false;
        used2[v] = num;
        for (int u : g[v]) {
            if (c[u] == -1) {
                c[u] = v;
                return true;
            }
        }
        for (int u : g[v]) {
            if (kuhn(c[u], num)) {
                c[u] = v;
                return true;
            }
        }
        return false;
    }

    void dfs(int v, int col, int &num) {
        part[v] = col;
        used[v] = true;
        if (col == 0) {
            kuhn(v, num++);
        }
        for (int u : g[v]) {
            if (!used[u])
                dfs(u, 1 - col, num);
        }
    }

    vector<pair<int, int>> solve() {
        int num = 1;
        for (int v = 0; v < n; v++) {
            if (!used[v])
                dfs(v, 0, num);
        }
        vector<pair<int, int>> ans;
        for (int i = 0; i < n; i++) {
            if (c[i] != -1)
                ans.emplace_back(c[i], i);
        }
        return ans;
    }

    void MIS_dfs(int v, vector<bool> &vis) {
        vis[v] = true;
        for (int u : g[v]) {
            if (vis[u]) continue;
            vis[u] = true;
            if (c[u] != -1 && !vis[c[u]])
                MIS_dfs(c[u], vis);
        }
    }

    vector<int> MIS() {
        vector<bool> vis(n, false);
        for (int v = 0; v < n; v++) {
            if (part[v] == 0) {
                bool is_used = false;
                for (int u : g[v]) {
                    if (c[u] == v)
                        is_used = true;
                }
                if (!is_used)
                    MIS_dfs(v, vis);
            }
        }

        vector<int> vertex_cover;
        vector<int> independent_set;
        for (int v = 0; v < n; v++) {
            if (part[v] == 0 && vis[v] || part[v] == 1 && !vis[v])
                independent_set.push_back(v);
```

```
            else
                vertex_cover.push_back(v);
        }
        return independent_set;
    }
};
```

3.8 online_bridges_search

graphs/online_bridges_search.cpp

```
struct DSU {
    int n, ans;
    vector<int> sz_cc, par_cc, par_2cc, par_t;
    vector<int> used;
    int t = 0;

    DSU(int n) : n(n) {
        ans = 0;
        sz_cc.resize(n, 1);
        par_cc.resize(n), iota(all(par_cc), 0);
        par_2cc.resize(n), iota(all(par_2cc), 0);
        par_t.resize(n), iota(all(par_t), 0);
        used.resize(n);
    }

    int get_comp(int v) {
        if (par_cc[v] == v)
            return v;
        return par_cc[v] = get_comp(par_cc[v]);
    }

    int get_main(int v) {
        if (par_2cc[v] == v)
            return v;
        return par_2cc[v] = get_main(par_2cc[v]);
    }

    void make_root(int v) {
        v = get_main(v);

        int last = v;
        v = get_main(par_t[v]);
        while (last != v) {
            int u = get_main(par_t[v]);
            par_t[v] = last;
            last = v;
            v = u;
        }
    }

    int get_lca(int v, int u) {
        t++;
        while (v != u) {
            if (v != -1 && used[v] == t)
                break;
            if (u != -1 && used[u] == t) {
                swap(v, u);
                break;
            }
            if (v != -1) used[v] = t;
            if (u != -1) used[u] = t;
            int pv = get_main(par_t[v]);
            v = v == -1 || pv == v ? -1 : pv;
            int pu = get_main(par_t[u]);
            u = u == -1 || pu == u ? -1 : pu;
        }
        return v;
    }

    void add_edge(int v, int u) {
        int cv = get_comp(v);
        int cu = get_comp(u);
        if (cv != cu) {//разные компоненты связности
            if (sz_cc[cv] < sz_cc[cu])
                swap(cv, cu);
            par_cc[cu] = cv;
            sz_cc[cv] += sz_cc[cu];

            cv = get_main(v);
            cu = get_main(u);
            make_root(cu);
            par_t[cu] = cv;
            ++ans;
            return;
        }
        cv = get_main(v);
        cu = get_main(u);
        if (cv == cu)//одна компонента рёберной двусвязности
            return;

        int lca = get_lca(cv, cu);
        while (cv != lca) {//сжатия вершин на пути от cv до cu в lca
            --ans;
            int p = get_main(par_t[cv]);
            par_2cc[cv] = lca;
            cv = p;
        }
        while (cu != lca) {
            --ans;
            int p = get_main(par_t[cu]);
            par_2cc[cu] = lca;
            cu = p;
        }
    }
};
```

3.9 search BCC

graphs/search BCC.cpp

```
//g[v] - list of adjacent vertices as [u, i] where i - idex of edge
//find vertex-biconnected components
vector<vector<pair<int, int>>> g;
vector<int> depth;
vector<bool> used;
vector<int> st;
vector<vector<int>> comp;

int dfs(int v, int p) {
    int mn = depth[v];
    int deg = 0;
    for (auto [u, i] : g[v]) {
        if (depth[u] == -1) {
            deg++;
            depth[u] = depth[v] + 1;
            st.push_back(i);
            int d = dfs(u, i);
            if (d >= depth[v]) {
                comp.emplace_back();
                while (true) {
                    auto edge = st.back();
                    st.pop_back();
                    comp.back().push_back(edge);
                    if (edge == i) break;
                }
            }
            mn = min(mn, d);
        } else {
            if (i != p && depth[u] <= depth[v])
                st.push_back(i);
            mn = min(mn, depth[u]);
        }
    }
    return mn;
}
```

3.10 search centroid

graphs/search centroid.cpp

```
vector<bool> used;
vector<int> sz;
vector<vector<int>> g;

void sizes(int v, int p) {
    sz[v] = 1;
    for (int u : g[v]) {
        if (u != p && !used[u])
            sizes(u, v), sz[v] += sz[u];
    }
}

int centroid (int v, int p, int n) {
    for (int u : g[v])
        if (u != p && !used[u] && sz[u] > n/2)
            return centroid(u, v, n);
    return v;
}
```

4 Теория чисел

4.1 Diophantine equation

number_theory/Diophantine equation.cpp

```
//find x and y such that ax+by=c, x->min and x >= 0
//if equation dont has solution solve_eq will return {-1, -1}
int expanded_gcd(int a, int b, int &x, int &y) {
    if (a == 0) {
        x = 0, y = 1;
        return b;
    }
    int g = expanded_gcd(b % a, a, x, y);
    int x2 = x;
    x = y - x * (b / a);
    y = x2;
    return g;
}

pair<ll, ll> solve_eq(int a, int b, int c) {
    int x = 0, y = 0;
    int g = expanded_gcd(abs(a), abs(b), x, y);
    if (abs(c) % g != 0)
        return mp(0, 0);
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    x *= c / g;
    y *= c / g;
    return mp((ll) x * (c / g), (ll) y * (c / g));
}
```

4.2 Expanded Euclidean algo

number_theory/Expanded Euclidean algo.cpp

```cpp
int expanded_gcd(int a, int b, int &x, int &y) {
    if (a == 0) {
        x = 0, y = 1;
        return b;
    }
    int g = expanded_gcd(b % a, a, x, y);
    int x2 = x;
    x = y - x * (b / a);
    y = x2;
    return g;
}
```

4.3 FFT double

number_theory/FFT double.cpp

```cpp
const ld eps = 1e-8;
const ld pi = acos(-1);

void fft(vector<complex<ld>> &A) {
    int n = A.size();
    assert((n & -n) == n);
    vector<int> rev(n);
    for (int i = 1; i < n; i++)
        rev[i] = (i & 1) * (n / 2) + rev[i / 2] / 2;
    for (int i = 0; i < n; i++) {
        if (i < rev[i])
            swap(A[i], A[rev[i]]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ld arg = (2 * pi) / len;
        complex<ld> w1{cos(arg), sin(arg)};
        for (int st = 0; st < n; st += len) {
            complex<ld> wi{1, 0};
            for (int i = st; i < st + len / 2; i++) {
                auto l = A[i], r = A[i + len / 2];
                A[i] = l + wi * r;
                A[i + len / 2] = l - wi * r;
                wi *= w1;
            }
        }
    }
}

vector<ll> mult(vector<int> A, vector<int> B) {
    int n = max(A.size(), B.size());
    while ((n & -n) != n) n += n & -n;
    n <<= 1;
    vector<complex<ld>> A2(n), B2(n);
    for (int i = 0; i < A.size(); i++)
        A2[i] = A[i];
    for (int i = 0; i < B.size(); i++)
        B2[i] = B[i];
    fft(A2);
    fft(B2);
    vector<complex<ld>> C2(n);
    for (int i = 0; i < n; i++)
        C2[i] = A2[i] * B2[i];
    fft(C2);
    vector<ll> C(n);
    for (int i = 0; i < n; i++) {
        assert(abs(C2[i].imag()) <= eps);
        C[i] = round(C2[i].real() / n);
    }
    reverse(C.begin() + 1, C.end());
    return C;
}
```

4.4 FFT modulo

number_theory/FFT modulo.cpp

```cpp
int power(int a, int b, const int mod) {
    int ans = 1;
    while (b) {
        if (b & 1)
            ans = (ll) ans * a % mod;
        a = (ll) a * a % mod;
        b >>= 1;
    }
    return ans;
}

void fft(vector<int> &A, const int mod) {
    int n = A.size();
    vector<int> rev(n);
    for (int i = 1; i < n; i++)
        rev[i] = (i & 1) * (n / 2) + rev[i / 2] / 2;
    for (int i = 0; i < n; i++) {
        if (i < rev[i])
            swap(A[i], A[rev[i]]);
    }
    const int g = 3;
    for (int len = 2; len <= n; len <<= 1) {
        int w1 = power(g, (mod - 1) / len, mod);
        for (int st = 0; st < n; st += len) {
            int wi = 1;
            for (int i = st; i < st + len / 2; i++) {
                int l = A[i], r = A[i + len / 2];
                A[i] = (l + (ll) r * wi) % mod;
                A[i + len / 2] = (l + (ll) (mod - wi) * r) % mod;
                wi = (ll) wi * w1 % mod;
            }
        }
    }
}

vector<int> mult(vector<int> A, vector<int> B) {
    int n = max(A.size(), B.size());
    while ((n & -n) != n) n += n & -n;
    n <<= 1;
    A.resize(n);
    B.resize(n);
    const int mod = (1 << 23) * 119 + 1;
    fft(A, mod);
    fft(B, mod);
    vector<int> C(n);
    for (int i = 0; i < n; i++)
        C[i] = (ll) A[i] * B[i] % mod;
    fft(C, mod);
    int rev_n = power(n, mod - 2, mod);
    for (int i = 0; i < n; i++)
        C[i] = (ll) C[i] * rev_n % mod;
    reverse(C.begin() + 1, C.end());
    return C;
}
```

4.5 floor sum

number_theory/floor sum.cpp

```cpp
//calcucate sum{0<=i<n} floor((k * i + b) / m)
int floor_sum(int k, int b, int m, int n) {
    int ans = 0;
    int tmp = k / m;
    ans += n * (n - 1) / 2 * tmp;
    ans += n * (b / m);
    k %= m;
    b %= m;
    if (k == 0)
        return ans;

    int ymax = (k * (n - 1) + b) / m;
    int res = (n - 1) * ymax;
    res -= floor_sum(m, m - b - 1, k, ymax);
    ans += res;
    return ans;
}
```

4.6 Long arithmetic (unfinished)

number_theory/Long arithmetic (unfinished).cpp

```cpp
//only +, +=, ==, <<, operations
struct BigInt {
    static const int base = 1e8;

    vector<int> ar;

    BigInt(int x = 0) {
        assert(x < base);
        ar.push_back(x);
    }

    friend BigInt operator+(const BigInt &lhs, const BigInt &rhs) {
        BigInt res; res.ar.pop_back();
        int i = 0, carry = 0;
        while (i < lhs.ar.size() || i < rhs.ar.size() || carry != 0) {
            ll sm = (ll) (i < lhs.ar.size() ? lhs.ar[i] : 0) + (i < rhs.ar.size() ? rhs.ar[i] : 0) + carry;
            res.ar.push_back(sm % base);
            carry = sm / base;
            i++;
        }
        return res;
    }

    friend BigInt& operator+=(BigInt &lhs, const BigInt &rhs) {
        return lhs = lhs + rhs;
    }

    friend bool operator==(const BigInt &, const BigInt &) = default;

    friend ostream& operator<<(ostream &o, BigInt x) {
        assert(!x.ar.empty());
        o << x.ar.back();
        for (int i = (int) x.ar.size() - 2; i >= 0; i--) {
            o << setfill('0') << setw(8) << x.ar[i];
        }
        return o;
    }
};
```

4.7 Miller-Rabin

number_theory/Miller-Rabin.cpp

```cpp
ll power(ll a, ll b, ll mod) {
    if (b == 0)
        return 1;
```

```
    else if (b % 2 == 0) {
        ll res = power(a, b / 2, mod);
        return (__int128) res * res % mod;
    }
    else
        return (__int128) a * power(a, b - 1, mod) % mod;
}

bool is_prime(ll n) {
    if (n == 1) return false;
    ll s = (n - 1) & -(n - 1);
    ll d = (n - 1) / s;
    for (ll a : {2, 7, 61, 2, 325, 9375, 28178, 450775, 9780504,
    1795265022}) {
        a %= n;
        if (a == 0) continue;
        if (gcd(a, n) != 1)
            return false;
        if (power(a, d, n) == 1) continue;
        bool is_p = false;
        for (ll i = 1; i <= s; i <<= 1) {
            if (power(a, i * d, n) == n - 1) {
                is_p = true;
                break;
            }
        }
        if (!is_p)
            return false;
    }
    return true;
}
```

4.8 ModInt

number_theory/ModInt.cpp

```
struct ModInt {
    static const int mod = 1e9 + 7;
    int val;

    ModInt(int val=0) : val(val) {}

    explicit operator int() {
        return val;
    }

    friend istream& operator>>(istream& i, ModInt &rhs) {
        return i >> rhs.val;
    }

    friend ostream& operator<<(ostream& o, const ModInt &rhs) {
        return o << rhs.val;
    }

    ModInt& operator+=(const ModInt& rhs) {
        val += rhs.val;
        val -= (val >= mod) * mod;
        return *this;
    }

    ModInt operator+(const ModInt& rhs) const {
        return ModInt(*this) += rhs;
    }

    ModInt& operator-=(const ModInt& rhs) {
        val -= rhs.val;
        val += (val < 0) * mod;
        return *this;
    }

    ModInt operator-(const ModInt& rhs) const {
        return ModInt(*this) -= rhs;
    }

    ModInt& operator*=(const ModInt& rhs) {
        val = (ll) val * rhs.val % mod;
        return *this;
    }

    ModInt operator*(const ModInt& rhs) const {
        return ModInt(*this) *= rhs;
    }

    ModInt operator/=(const ModInt& rhs) {
        *this *= rhs.power(mod - 2);
        return *this;
    }

    ModInt operator/(const ModInt& rhs) const {
        return ModInt(*this) /= rhs;
    }

private:
    ModInt power(int ind) const {
        ModInt res = 1;
        ModInt p = *this;
        while (ind) {
            if (ind & 1)
                res *= p;
            p *= p;
            ind >>= 1;
        }
        return res;
    }
};
```

4.9 Pollard

number_theory/Pollard.cpp

```
void pollard(ll n, vector<ll> &fact) {
    assert(n >= 2);
    if (n <= 100) {
        int i = 2;
        while (i * i <= n) {
            while (n % i == 0) {
                fact.push_back(i);
                n /= i;
            }
            i++;
        }
        if (n != 1)
            fact.push_back(n);
        return;
    }
    if (is_prime(n)) {
        fact.push_back(n);
        return;
    }

    auto f = [n](ll x)->ll {
        return ((__int128) (x + 1) * (x + 1)) % n;
    };

    while (true) {
        ll x = gen() % (n - 1) + 1;
        ll y = f(x);
        ll g = gcd(abs(x - y), n);
        int it = 0;
        while ((g <= 1 || g == n) && it < min<ll>(1e3, n + 5)) {
            x = f(x);
            y = f(f(y));
            g = gcd(abs(x - y), n);
            ++it;
        }
        if (g <= 1 || g == n) continue;
        pollard(g, fact);
        pollard(n / g, fact);
        break;
    }
}
```

5 Геометрия

5.1 build tangents parallel line

geometry/build tangents parallel line.cpp

```
bool comp(pt p1, pt p2) {//compares two vectors on polar angle
    pair<int, int> pr1{0, 1}, pr2{1, 0};
    int hp1 = mp(p1.y, p1.x) < mp(0ll, 0ll);
    int hp2 = mp(p2.y, p2.x) < mp(0ll, 0ll);
    if (hp1 != hp2)
        return hp1 < hp2;
    return cross(p1, p2) > 0;
}

pair<int, int> find_tangent(vector<pt> P, pt p) {
    vector<pair<pt, int>> sides;
    for (int i = 0; i < (int) P.size(); i++)
        sides.emplace_back(P[(i + 1) % P.size()] - P[i], i);
    sort(all(sides), [](auto i, auto j){return comp(i.first, j.
    first);});

    vector<int> answer;
    int idx1 = lower_bound(all(sides), mp(p, -1), [](auto i, auto
    j){return comp(i.first, j.first);})
        - sides.begin();
    idx1 %= sides.size();
    int idx2 = lower_bound(all(sides), mp(pt(0, 0) - p, 1), [](
    auto i, auto j){return comp(i.first, j.first);})
        - sides.begin();
    idx2 %= sides.size();

    return mp(sides[idx1].second, sides[idx2].second);
}
```

5.2 build tangents through point

geometry/build tangents through point.cpp

```
pair<int, int> find_tangent(vector<pt> &P, pt p) {
    int mx = 0, mn = 0;
    int n = P.size();

    auto sign = [&](int i, int j) {
        auto res = cross(P[j] - p, P[i] - p);
        if (res == 0)
            return 0;
        return res > 0 ? 1 : -1;
    };

    for (int i = (1 << 20); i >= 1; i >>= 1) {
        int c1 = (mx + i) % n, c2 = ((mx - i) % n + n) % n;
        if (sign(c1, mx) > 0)
```

```
            mx = c1;
        if (sign(c2, mx) > 0)
            mx = c2;

        c1 = (mn + i) % n, c2 = ((mn - i) % n + n) % n;
        if (sign(c1, mn) < 0)
            mn = c1;
        if (sign(c2, mn) < 0)
            mn = c2;
    }
    return mp(mx, mn);
}
```

5.3 convex hull v2

geometry/convex hull v2.cpp

```
vector<pt> convex_hull(vector<pt> P) {
    int n = P.size();
    assert(n > 0);
    pt p = P[0];
    for (int i = 1; i < n; i++)
        p = min(p, P[i]);

    sort(all(P), [&](auto a, auto b) {
        pt pa = a - p;
        pt pb = b - p;

        ll an = cross(pa, pb);
        if (an == 0)
            return len(pa) < len(pb);
        return an > 0;
    });

    vector<pt> ch;
    int sz = 0;
    for (auto i : P) {
        while ((sz >= 1 && i == ch.back()) || (sz >= 2 && cross(ch[sz - 1] - ch[sz - 2], i - ch[sz - 1]) <= 0)) {
            ch.pop_back();
            sz--;
        }
        ch.push_back(i);
        sz++;
    }
    return ch;
}
```

5.4 convex hull

geometry/convex hull.cpp

```
vector<pt> convex_hull(vector<pt> P) {
    assert(!P.empty());
    pt A = P[0], B = P[0];
    for (pt p : P) {
        A = min(A, p);
        B = max(B, p);
    }
    if (A == B)
        return {A};
    vector<pt> up, down;
    for (pt p : P) {
        if (cross(B - A, p - A) >= 0)
            up.push_back(p);
        if (cross(B - A, p - A) <= 0)
            down.push_back(p);
    }
    sort(all(up));
    sort(rall(down));
    vector<pt> st_up, st_down;
    for (pt p : up) {
        int sz = st_up.size();
        while ((sz >= 1 && st_up.back() == p) || (sz >= 2 && cross(st_up[sz - 1] - st_up[sz - 2], p - st_up[sz - 1]) >= 0))
            st_up.pop_back(), sz--;
        st_up.push_back(p);
    }
    for (pt p : down) {
        int sz = st_down.size();
        while ((sz >= 1 && st_down.back() == p) || (sz >= 2 && cross(st_down[sz - 1] - st_down[sz - 2], p - st_down[sz - 1]) >= 0))
            st_down.pop_back(), sz--;
        st_down.push_back(p);
    }
    st_up.pop_back();
    st_down.pop_back();
    for (pt p : st_down)
        st_up.push_back(p);
    reverse(all(st_up));
    return st_up;
}
```

5.5 is point in polygon

geometry/is point in polygon.cpp

```
bool in_polygon(vector<pt> &P, pt p) {
    int n = P.size();
    if (cross(P[1] - P[0], p - P[0]) < 0 || cross(P[0] - P[n - 1], p - P[n - 1]) < 0)// <= 0 - if pt cant be on side
        return false;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int m = (l + r) / 2;
        if (cross(P[m] - P[0], p - P[0]) > 0)
            l = m;
        else
            r = m;
    }
    return cross(P[r] - P[l], p - P[l]) >= 0;// > 0 - if pt cant be on side
}
```

5.6 Minkovski

geometry/Minkovski.cpp

```
vector<pt> minkovski(vector<pt> A, vector<pt> B) {
    vector<pt> vec;
    pt p1 = A[0], p2 = B[0];
    for (int i = 0; i < A.size(); i++) {
        vec.emplace_back(A[i] - A[(i + 1) % A.size()]);
        //p1 = max(p1, A[i]);
        if (A[i].y > p1.y || A[i].y == p1.y && A[i].x < p1.x)
            p1 = A[i];
    }
    for (int i = 0; i < B.size(); i++) {
        vec.emplace_back(B[i] - B[(i + 1) % B.size()]);
        //p2 = max(p2, B[i]);
        if (B[i].y > p2.y || B[i].y == p2.y && B[i].x < p2.x)
            p2 = B[i];
    }
    sort(all(vec), [](auto i, auto j) {
        ld an1 = atan2(cross({1, 0}, i), scalar({1, 0}, i));
        ld an2 = atan2(cross({1, 0}, j), scalar({1, 0}, j));
        return an1 < an2;
    });

    pt cur = p1 + p2;
    vector<pt> res{cur};
    for (int i = 0; i < vec.size(); i++) {
        cur = cur + vec[i];
        res.push_back(cur);
    }
    return res;
}
```

6 Строки

6.1 Aho-Corasick

strings/Aho-Corasick.cpp

```
int ALPH = 26;

struct AhoCorasick {
    struct node {
        vector<int> to;
        vector<bool> trie;
        bool ter = false;
        int link = -1;
        int p = -1;
        int pc = -1;

        int suplink = -1;

        node() {
            to.resize(ALPH, -1);
            trie.resize(ALPH, false);
        }
    };

    vector<node> mem;

    AhoCorasick() {
        mem.resize(1);
    }

    void add_string(vector<int> s) {
        int v = 0;
        for (int c : s) {
            if (!mem[v].trie[c]) {
                int u = mem.size();
                mem[v].to[c] = u;
                mem.emplace_back();
                mem[u].p = v;
                mem[u].pc = c;
                mem[v].trie[c] = true;
            }
            v = mem[v].to[c];
        }
        mem[v].ter = true;
    }

    void build_automaton() {
        queue<int> Q;
        Q.push(0);
        while (!Q.empty()) {
            int v = Q.front();
            Q.pop();

            if (v == 0 || mem[v].p == 0)
                mem[v].link = 0;
            else
```

```
                mem[v].link = mem[mem[mem[v].p].link].to[mem[v].pc];

            mem[v].suplink = mem[v].link;
            if (!mem[mem[v].link].ter)
                mem[v].suplink = mem[mem[v].link].suplink;

            for (int c = 0; c < ALPH; c++) {
                if (mem[v].trie[c])
                    Q.push(mem[v].to[c]);
                else if (v == 0)
                    mem[v].to[c] = 0;
                else
                    mem[v].to[c] = mem[mem[v].link].to[c];
            }
        }
    }
};
```

6.2 hashes

strings/hashes.cpp

```
const int maxn = 1e5 + 10;

struct ModInt {
    static const int base1 = 29, mod1 = 1e9 + 7;
    static const int base2 = 31, mod2 = 999973423;//1e9 + 9;

    int val1, val2;

    ModInt(int val1=0, int val2=0) : val1(val1), val2(val2) {}

    friend ModInt operator+(const ModInt &a, const ModInt &b) {
        ModInt res(a.val1 + b.val1, a.val2 + b.val2);
        res.val1 -= (res.val1 >= ModInt::mod1) * ModInt::mod1;
        res.val2 -= (res.val2 >= ModInt::mod2) * ModInt::mod2;
        return res;
    }

    friend ModInt operator-(const ModInt &a, const ModInt &b) {
        ModInt res(a.val1 - b.val1, a.val2 - b.val2);
        res.val1 += (res.val1 < 0) * ModInt::mod1;
        res.val2 += (res.val2 < 0) * ModInt::mod2;
        return res;
    }

    friend ModInt operator*(const ModInt &a, const ModInt &b) {
        ModInt res(((ll) a.val1 * b.val1) % ModInt::mod1, ((ll) a.val2 * b.val2) % ModInt::mod2);
        return res;
    }

    friend bool operator==(const ModInt &a, const ModInt &b) {
        return a.val1 == b.val1 && a.val2 == b.val2;
    }

    friend bool operator<(const ModInt &a, const ModInt &b) {
        if (a.val1 == b.val1)
            return a.val2 < b.val2;
        return a.val1 < b.val1;
    }
};

array<ModInt, maxn> pows;

vector<ModInt> build_pref_hashes(string &s) {
    int n = (int) s.size();
    vector<ModInt> P(n);
    ModInt h;
    for (int i = 0; i < n; i++) {
        h = h * ModInt(ModInt::base1, ModInt::base2) + ModInt(s[i] - 'a' + 1);
        P[i] = h;
    }
    return P;
}

vector<ModInt> build_suf_hashes(string &s) {
    int n = (int) s.size();
    vector<ModInt> S(n);
    ModInt h;
    for (int i = n - 1; i >+ 0; i--) {
        h = h * ModInt(ModInt::base1, ModInt::base2) + ModInt(s[i] - 'a' + 1);
        S[i] = h;
    }
    return S;
}

ModInt get_hash(int l, int r, vector<ModInt> &A, int flag=0) {// flag = 0 -> straight hash, flag != 0 -> reverse hash; return hash substring[l:r]
    if (flag == 0) {
        if (l == 0)
            return A[r];
        return A[r] - A[l - 1] * pows[r - l + 1];
    }
    if (r + 1 == A.size())
        return A[l];
    return A[l] - A[r + 1] * pows[r - l + 1];
}

void precalc() {
    pows[0] = 1;
    for (int i = 1; i < pows.size(); i++)
        pows[i] = pows[i - 1] * ModInt(ModInt::base1, ModInt::base2);
}
```

6.3 Manaker (may have mistakes)

strings/Manaker (may have mistakes).cpp

```
//sample: s = aaba
//      : A = 1121 - odd palindrome
//      : B = 0100 - even palindrome
void manaker(const string &s) {
    int n = s.size();
    vector<int> A(n);
    int l = -1, r = -1;
    for (int i = 0; i < n; i++) {
        if (i <= r)
            A[i] = min(A[l + r - i], r - i + 1);
        while (i - A[i] >= 0 && i + A[i] < n && s[i - A[i]] == s[i + A[i]])
            A[i]++;
        if (i + A[i] - 1 > r) {
            l = i - A[i] + 1;
            r = i + A[i] - 1;
        }
    }
    vector<int> B(n);
    l = -1, r = -1;
    for (int i = 0; i < n; i++) {
        if (i <= r)
            B[i] = min(B[l + r - i + 1], r - i + 1);
        while (i - B[i] - 1 >= 0 && i + B[i] < n && s[i - B[i] - 1] == s[i + B[i]])
            B[i]++;
        if (i + B[i] - 1 > r) {
            l = i - B[i];
            r = i + B[i] - 1;
        }
    }
}
```

6.4 suffix array

strings/suffix array.cpp

```
vector<int> suffix_array(string &s) {
    s += '#';
    int n = s.size();
    vector<int> P(n);
    iota(all(P), 0);
    sort(all(P), [&s](int i, int j){return s[i] < s[j];});
    vector<int> C(n);
    for (int i = 1; i < n; i++)
        C[P[i]] = C[P[i - 1]] + (s[P[i]] != s[P[i - 1]]);
    for (int len = 1; len < n; len <<= 1) {
        vector<int> cnt(n);
        for (int i = 0; i < n; i++) {
            P[i] = (P[i] - len + n) % n;
            cnt[C[P[i]]]++;
        }
        for (int i = 1; i < n; i++)
            cnt[i] += cnt[i - 1];
        vector<int> P2(n), C2(n);
        for (int i = n - 1; i >= 0; i--) {
            P2[cnt[C[P[i]]] - 1] = P[i];
            cnt[C[P[i]]]--;
        }
        for (int i = 1; i < n; i++) {
            auto k1 = mp(C[P2[i]], C[(P2[i] + len) % n]);
            auto k2 = mp(C[P2[i - 1]], C[(P2[i - 1] + len) % n]);
            C2[P2[i]] = C2[P2[i - 1]] + (k1 != k2);
        }
        swap(P, P2);
        swap(C, C2);
    }
    P.erase(P.begin());
    s.pop_back();
    return P;
}
```

6.5 suffix automaton

strings/suffix automaton.cpp

```
const int ALPH = 26;

struct SA {
    struct Node {
        int to[ALPH];
        int link = -1;
        int len = 0;

        Node() {
            fill(to, to + ALPH, -1);
        }
    };

    vector<Node> mem;
    int head = 0;
```

```
16
17      SA() {
18          mem.resize(1);
19      }
20
21      void extend(int c) {
22          int new_head = mem.size();
23          mem.emplace_back();
24          mem[new_head].len = mem[head].len + 1;
25          for (; head != -1; head = mem[head].link) {
26              if (mem[head].to[c] == -1) {
27                  mem[head].to[c] = new_head;
28                  continue;
29              }
30              if (mem[head].len + 1 == mem[mem[head].to[c]].len) {
31                  mem[new_head].link = mem[head].to[c];
32                  break;
33              }
34              int v = mem[head].to[c];
35              int vc = mem.size();
36              mem.push_back(mem[v]);
37              mem[v].link = vc;
38              mem[vc].len = mem[head].len + 1;
39              while (head != -1 && mem[head].to[c] == v) {
40                  mem[head].to[c] = vc;
41                  head = mem[head].link;
42              }
43              mem[new_head].link = vc;
44              break;
45          }
46          if (mem[new_head].link == -1)
47              mem[new_head].link = 0;
48          head = new_head;
49      }
50  };
```

6.6 Z and prefix functions

strings/Z and prefix functions.cpp

```
1  vector<int> PrefFunc(string &s) {
2      int n = s.size();
3      vector<int> P(n);
4      for (int i = 1; i < n; i++) {
5          int j = P[i - 1];
6          while (j != 0 && s[j] != s[i]) {
7              j = P[j - 1];
8          }
9          if (s[j] == s[i])
10             j++;
11         P[i] = j;
12     }
13     return P;
14 }
15
16 vector<int> ZFunc(string &s) {
17     int n = s.size();
18     vector<int> Z(n);
19     int l = -1, r = -1;
20     for (int i = 1; i < n; i++) {
21         if (r >= i)
22             Z[i] = min(Z[i - l], r - i + 1);
23         while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]])
24             Z[i]++;
25         if (i + Z[i] - 1 >= r) {
26             l = i;
27             r = i + Z[i] - 1;
28         }
29     }
30     return Z;
31 }
```

7 Разное

7.1 otjig

other/otjig.cpp

```
1  //sample using otjig method
2  //statment: you need to find placement N
3  //queens on chess board(N*N) that queens
4  //don't attack each other
5
6  mt19937 gen(1984);
7
8  bool check(vector<int> &P) {
9      for (int i = 0; i < P.size(); i++) {
10         for (int j = i + 1; j < P.size(); j++) {
11             if (i + P[i] == j + P[j] || i - P[i] == j - P[j])
12                 return false;
13         }
14     }
15     return true;
16 }
17
18 void solve() {
19     int n;
20     cin >> n;
21
22     vector<int> P(n);
23     iota(all(P), 0);
24     shuffle(all(P), gen);
25     vector<int> D1(n * 2 - 1), D2(n * 2 - 1);
26
27     for (int i = 0; i < n; i++) {
28         D1[i + P[i]]++;
29         D2[i - P[i] + (n - 1)]++;
30     }
31
32     auto calc_score = [&](){
33         int score = 0;
34         for (int i = 0; i < n; i++) {
35             for (int j = i + 1; j < n; j++)
36                 score += (P[i] + i == P[j] + j) || (P[i] - i == P[
    j] - j);
37         }
38         return score;
39     };
40
41     auto apply = [&](int i, int j, int score){
42         auto calc_d = [&P, n](int i){
43             return mp(i + P[i], i - P[i] + (n - 1));
44         };
45         vector<int> ar_d1, ar_d2;
46         auto calc_subscore = [&](){
47             int res = 0;
48             for (auto d1 : ar_d1)
49                 res += D1[d1] * (D1[d1] - 1) / 2;
50             for (auto d2 : ar_d2)
51                 res += D2[d2] * (D2[d2] - 1) / 2;
52             return res;
53         };
54         for (int tmp = 0; tmp < 2; tmp++) {
55             auto [d1i, d2i] = calc_d(i);
56             auto [d1j, d2j] = calc_d(j);
57             ar_d1.push_back(d1i), ar_d1.push_back(d1j);
58             ar_d2.push_back(d2i), ar_d2.push_back(d2j);
59             swap(P[i], P[j]);
60         }
61         ar_d1.erase(unique(all(ar_d1)), ar_d1.end());
62         ar_d2.erase(unique(all(ar_d2)), ar_d2.end());
63         score -= calc_subscore();
64         {
65             auto [d1i, d2i] = calc_d(i);
66             auto [d1j, d2j] = calc_d(j);
67             D1[d1i]--, D1[d1j]--;
68             D2[d2i]--, D2[d2j]--;
69         }
70         swap(P[i], P[j]);
71         {
72             auto [d1i, d2i] = calc_d(i);
73             auto [d1j, d2j] = calc_d(j);
74             D1[d1i]++, D1[d1j]++;
75             D2[d2i]++, D2[d2j]++;
76         }
77         score += calc_subscore();
78         return score;
79     };
80
81     int score = 0;
82     for (int i = 0; i < 2 * n - 1; i++)
83         score += D1[i] * (D1[i] - 1) / 2 + D2[i] * (D2[i] - 1) /
    2;
84     double T = 1;
85     double c = 0.999;
86     uniform_real_distribution<double> dis(0, 1);
87     while (score != 0) {
88         int i = gen() % n, j = gen() % n;
89         if (i == j) continue;
90
91         swap(P[i], P[j]);
92         int real_new_score = calc_score();
93         swap(P[i], P[j]);
94
95         int new_score = apply(i, j, score);
96
97         int d = new_score - score;
98         if (new_score < score || dis(gen) < exp(-d / T))
99             score = new_score;
100        else {
101            apply(i, j, new_score);
102        }
103        T *= c;
104    }
105    assert(check(P));
106    rep(n) {
107        cout << P[i] + 1 << '\n';
108    }
109 }
```

7.2 two sat

other/two sat.cpp

```
1  struct TwoSat {
2      vector<vector<int>> g, gr;
3      vector<int> comp, order;
4      vector<bool> used;
5      int n;
6
7      TwoSat(int n) : n(n) {
8          g.resize(n * 2), gr.resize(n * 2);
9          comp.resize(n * 2), used.resize(n * 2);
10     }
```

```
11
12 private:
13     void add_edge(int v, int u) {
14         g[v].push_back(u);
15         gr[u].push_back(v);
16     }
17
18 public:
19     void add_or(int v, int u) {
20         add_edge(v ^ 1, u);
21         add_edge(u ^ 1, v);
22     }
23
24     vector<int> get() {
25         auto dfs = [&](auto &&self, int v)->void {
26             used[v] = true;
27             for (int u : g[v]) {
28                 if (!used[u])
29                     self(self, u);
30             }
31             order.push_back(v);
32         };
33         for (int v = 0; v < 2 * n; v++) {
34             if (!used[v])
35                 dfs(dfs, v);
36         }
37         reverse(all(order));
38         auto dfs2 = [&](auto &&self, int v, int c)->void {
39             comp[v] = c;
40             for (int u : gr[v]) {
41                 if (comp[u] == 0)
42                     self(self, u, c);
43             }
44         };
45         int c = 0;
46         for (int v : order) {
47             if (comp[v] == 0)
48                 dfs2(dfs2, v, ++c);
49         }
50         vector<int> ans(n);
51         for (int v = 0; v < 2 * n; v += 2) {
52             if (comp[v] == comp[v ^ 1]) {
53                 fill(all(ans), -1);
54                 return ans;
55             }
56             if (comp[v] > comp[v ^ 1])
57                 ans[v >> 1] = 1;
58         }
59         return ans;
60     }
61 };
```